GUÍA COMPLETA DE SUSTENTACIÓN Y CONTEXTO

Semanas 9 y 10 · Ingeniería del Software III · Proyecto: SIGEPAC (Grupo 4)

Universidad de Pamplona — Documento de apoyo rápido para defensa y examen

PARTE 1: LO QUE TIENES QUE SUSTENTARLE AL PROFESOR

Resumen coherente, claro y no técnico con los resultados de cada entrega:

ENTREGA SEMANA 9: Taller 9 y Comparativo

¿De qué se trataba?

Demostrar con el código real de nuestro proyecto qué le pasa al sistema cuando el cliente pide cambios, y cómo cambia el costo si trabajamos en Cascada, en RUP o con nuestra Arquitectura Hexagonal.

¿Qué hicimos?

Simulamos 3 cambios reales sobre nuestro servicio de la plaza cívica (domain-service-sigepac):

- **Cambio 1 (Regla de negocio):** Dar un 10% de descuento si el inquilino paga temprano (pronto pago).
- **Cambio 2 (Infraestructura):** Cambiar la base de datos de pruebas (H2) a una base de datos real (PostgreSQL/MySQL).
- **Cambio 3 (Alcance):** Clasificar los locales por categoría (comida, artesanías, etc.) y registrar qué administrador hizo el trámite.

¿Cuáles fueron los resultados que obtuvimos? (Mostrar al profesor):

- **En el Cambio 1 (Descuento):** Se modificaron 9 archivos, pero todos quedaron concentrados en el dominio y la aplicación. La base de datos y la web no se enteraron. Dolió un poco en las pruebas unitarias porque tocó recalcular valores esperados a mano.
- **En el Cambio 2 (Base de datos):** CERO archivos de código Java cambiaron. Solo tocamos 2 archivos de configuración (pom.xml y application-prod.yml). Demostró que la Arquitectura Hexagonal funcionó perfecto: al negocio no le importa qué motor de base de datos existe.
- **En el Cambio 3 (Categorías):** Se tocaron 10 archivos de arriba a abajo, pero de forma ordenada gracias a que cada caso de uso está separado e independiente.

> **[!] La Conclusión que le dices al profesor en Semana 9:**
> "Profesor, la conclusión del taller es que la metodología decide cuándo nos enteramos del cambio y cuánto papeleo cuesta (en Cascada es una pesadilla reabrir fases; en RUP se mete en la siguiente iteración). Pero la arquitectura es la que decide cuántos archivos tocamos realmente en el código".

ENTREGA SEMANA 10: Deuda Técnica, Spikes y Sprints

¿De qué se trataba?

Dejar de ver el proyecto como un código perfecto y ser autocríticos: identificar qué cosas dejamos pendientes o hechas 'de afán' para poder entregar (deuda técnica), investigar dudas antes de programar (spikes) y organizar la solución en 2 iteraciones en Trello.

¿Cuáles fueron los resultados que obtuvimos?:

- **1. Seis Deudas Técnicas reales:** La principal: la BD se crea sola con Hibernate (ddl-auto: update), lo cual es riesgoso en producción si se borran datos. Otras: falta de paginación en historial extenso y falta de Swagger para el frontend.
- **2. Dos Spikes formulados:** Spike 1 (2 horas): Averiguar si sirve más Flyway o Liquibase para migraciones. Spike 2 (3 horas): Investigar cómo bloquear registros para evitar que dos cajeros cobren la misma deuda al mismo tiempo.
- **3. Plan de 2 Sprints en Trello:** Sprint 1 (Base técnica firme): Ítems urgentes (A-01 a A-03) en 'Iteración actual' (pruebas en verde y Flyway). Sprint 2 (Calidad): Tareas futuras (A-04 a A-06) en 'Backlog' (paginación, Swagger y seguridad).
- **4. Definición de Hecho (DoD):** Seis reglas que debe cumplir toda tarea para darse por terminada (pruebas pasando, revisión

de pares y README actualizado).

[>] Resumen en una sola frase por si te pide ser breve:

- Semana 9: Demostramos que la arquitectura hexagonal aisló el negocio de la BD (0 clases Java modificadas).
- Semana 10: Registramos 6 deudas reales, 2 spikes con timebox y organizamos 2 sprints en Trello.

PARTE 2: QUÉ HICIMOS CON LAS GUÍAS, QUÉ SE ALTERÓ Y CONTEXTO

1. Las guías y materiales que se usaron

- **Guía de estudio oficial (Sem. 9 y 10):** Teoría de Cascada (Royce), RUP (Kruchten), Scrum, Cuadrante de Fowler y rúbricas.
- **Plantilla del Taller 9 ('El cambio de alcance'):** Hoja oficial con las 3 fichas para los cambios.
- **Ejemplo de la Ferretería:** Matriz modelo comparativa Hexagonal vs Capas y Cascada vs RUP.

2. ¿Qué fue lo que se hizo exactamente?

- **Taller_9_Ficha_Cambio_Alcance.docx:** Diligenciado con las clases reales de SIGEPAC para los 3 cambios (Pronto pago, BD PostgreSQL, Categoría de locales).
- **Semana_9_Comparativo_Clasico_Actual.docx:** Informe formal de 4 páginas con mapeo a las 4 fases de RUP, comparación de cambios y autocrítica del Grupo 4.
- **Semana_10_Backlog_Arquitectonico.docx:** 6 deudas técnicas en cuadrante de Fowler, 2 Spikes con timebox, Backlog (escalas 1-10 y 10-100), plan de 2 Sprints y DoD.
- **Sincronización en Trello (SIGEPAC · XP):** Tarjetas A-01 a A-03 y SP-01/SP-02 en 'Iteración actual' (Sprint 1), y A-04 a A-06 en 'Historias (backlog)' (Sprint 2).

3. ¿Qué se alteró en tu proyecto?

- **Cero código Java modificado:** No se alteró ninguna clase en src/main/ ni src/test/. Las 70 pruebas unitarias siguen en verde.
- **Cero bases de datos tocadas:** No se modificó ni borró ningún dato existente.
- **Solo archivos nuevos de documentación:** Todo se generó de forma limpia en la carpeta docs/entregas/.

4. ¿Qué debes presentarle al profesor exactamente?

• Para Semana 9: El Word Taller_9_Ficha_Cambio_Alcance.docx y el informe formal Semana_9_Comparativo_Clasico_Actual.docx.
• Para Semana 10: El Word Semana_10_Backlog_Arquitectonico.docx y 2 capturas de Trello (vista global del tablero y detalle del Spike SP-01 con su checklist).

5. 'Machete': Preguntas típicas de sustentación

Q: ¿Por qué el cambio de BD (H2 a PostgreSQL) no costó nada en su código?
R: Porque gracias a la Arquitectura Hexagonal y la Inversión de Dependencias (DIP), el dominio no sabe qué BD existe. Solo se tocaron 2 archivos de configuración (application-prod.yml y pom.xml); ninguna clase de negocio cambió.

Q: ¿Qué es una tarjeta A-00 y una SP-00 en su Trello?
R: Las HU son historias de usuario. Las 'A' son Ítems Arquitectónicos (tareas técnicas de soporte/migración). Las 'SP' son Spikes, investigaciones técnicas acotadas en tiempo (Timebox) para despejar dudas antes de codificar.

Q: ¿Qué es el Cuadrante de Fowler y denme un ejemplo de su proyecto?
R: Clasifica la deuda técnica en Deliberada/Inadvertida y Prudente/Imprudente. Ejemplo: ddl-auto: update fue Deliberada-Prudente (permitió modelar rápido el prototipo sabiendo que luego se pagaría con Flyway en el Sprint 1).

Q: ¿Cuál es la conclusión central entre Metodología y Arquitectura?
R: La arquitectura decide cuántos archivos tocamos; la metodología decide cuándo descubrimos el cambio y cuánto retrabajo burocrático cuesta.